

Kunai: A Verifier-Safe Layered DSL for Multi-Encapsulation Packet Filtering on XDP

Anonymous Author(s)

ABSTRACT

Packet capture and network debugging require selecting the packets of interest early, inside the Linux kernel. Yet `pcap-filter`, the language behind `tcpdump`, has no syntax for multi-level encapsulation, repeated occurrences of the same protocol, or the variable-length lists and options whose field offsets change from packet to packet. Running a filter in the kernel additionally requires emitting bytecode that the Linux eBPF verifier accepts, and a naive lowering of such filters is rejected.

We present KUNAI, a packet-filter DSL together with a compiler that lowers its expressions to eBPF bytecode. A KUNAI expression describes a packet's layer structure and field conditions in a single expression, and resolves each field's offset and type from protocol definitions written in a subset of P4. The compiler records every header's start offset at run time, bounds each variable-length scan by a compile-time constant, and places a verifier-checkable bounds check before every packet read.

We evaluate KUNAI on filters spanning GTP-U, SRv6, Geneve, and TCP options. The generated bytecode is accepted by the verifier across six kernels (Linux 6.1–7.0) on both the XDP and tc hosts, matches and rejects the intended packets, and runs within a few percent of `pcap-filter` per packet. KUNAI thus makes filters that `pcap-filter` cannot express executable inside the Linux kernel.

1 INTRODUCTION

Packet capture is a basic operation in debugging and fault analysis of network systems. As link bandwidth and traffic volume grow, forwarding every packet to user space for storage no longer scales, so a capture tool must select only the packets relevant to its purpose early, inside the kernel. The de-facto language for this selection has been `tcpdump/libpcap's pcap-filter` [15, 21].

`pcap-filter` concisely expresses conditions on L2–L4 header fields. It has, however, no syntax for multi-level encapsulation, repeated occurrences of the same protocol, or lists and options whose length varies from packet to packet. In the inner IPv4 of a GTP-U or Geneve tunnel, an SRv6 segment list, or TCP options, the offset of the target field changes per packet [1, 7, 8, 10]. Filter languages that can follow these structures by field name do exist — the Wireshark display filter [23] is one — but they evaluate after capture, in user space, and cannot run early in the kernel data path. User studies of packet-analysis tools likewise report protocol coverage and the expressiveness of the filter language as open problems [20].

Closing this gap requires meeting two requirements at once. First, the filter language must express layer structure, repeated protocols, and variable-length lists and options syntactically; without such syntax, the user is left to derive byte-offset arithmetic by hand.

Second, to run the filter inside the Linux kernel, the compiler must emit eBPF bytecode that the verifier accepts [5, 9, 14].

We present KUNAI, a filter DSL and compiler that meets both requirements. KUNAI describes a packet's layer structure as a *layer-chain* and resolves the offset and type of each field from protocol definitions written in a subset of P4 [17], then lowers the filter expression to eBPF bytecode. A naive lowering of a filter that contains variable-length structure is rejected by the eBPF verifier, so transforming it into an accepted form is the central problem of this paper. KUNAI performs this transformation by recording each header's start offset at run time, bounding every variable-length scan by a compile-time constant, and placing a verifier-checkable bounds check before every packet read; we describe the code generation in §4.

This paper describes the design and code generation of KUNAI and evaluates it on filters that contain multi-level encapsulation and variable-length structure. We show that filters spanning GTP-U and SRv6 are expressible in KUNAI, that the generated bytecode is accepted by the verifier across six kernels (Linux 6.1–7.0) on both the XDP and tc hosts, that it matches and rejects packets as intended, and that its per-packet cost is comparable to `pcap-filter`.

2 RELATED WORK

`tcpdump/libpcap's pcap-filter` is a filter language that begins with the Packet Filter [16] and the BSD Packet Filter [15], and it expresses L2–L4 header conditions concisely and runs them inside the kernel. It has, however, no syntax for distinguishing repeated occurrences of the same protocol, or for walking by field name the elements whose position varies per packet, such as an SRv6 segment list or TCP options. The more expressive Wireshark display filter [23] references these fields by name, but is limited to userspace evaluation over already-captured packets and cannot run in the kernel data path. No existing filter language thus provides both field-name expressiveness and in-kernel execution.

Prior work on encapsulation-aware filters includes NetPDL/NetPFL and pFSA/xpFSA. NetPDL/NetPFL connect a description of protocol headers with tunnel-aware filter expressions [3, 19]. pFSA/xpFSA represent the order in which headers appear within an encapsulation as state transitions and formalize the composition of filters and field access [2, 12]. These share our problem setting in bringing multi-level encapsulation into the filter's expression. They do not, however, emit eBPF bytecode loadable on the Linux kernel data path, and so do not provide the in-kernel execution this paper presupposes.

Among combinations of P4 and eBPF, `p4c-ebpf` and `p4c-xdp` compile a P4 data-plane program to Linux eBPF / XDP / tc [18, 22]. These are compilers that move an entire P4 pipeline to an eBPF target; they are not a filter language that returns per-packet match/reject from a filter expression. KUNAI differs from them in

using P4 only as a protocol definition rather than as a data-plane language [17].

The work above has separately addressed the execution mechanism of packet filters, the expression of filters over multi-level encapsulation, and the compilation of P4 programs to eBPF. When a filter that contains multi-level encapsulation or variable-length structure is lowered to eBPF, however, **the start offset of a following header changes per packet, so the eBPF verifier cannot statically confirm the range of the generated packet reads and a naive lowering is rejected**. The contribution of this paper is to realize this lowering in a form the eBPF verifier accepts. Concretely, a code generation that switches to a bounds-checked read according to whether the base of an offset is a compile-time constant or a range that changes at run time lowers these filters into a verifier-accepted form, and we generalize this transformation across the protocols declared in the P4 subset.

3 DESIGN

3.1 Overview

Figure 1 shows the overall structure of KUNAI. The KUNAI compiler takes two inputs. The first is a KUNAI DSL expression, the filter condition written by the user. The second is a set of protocol definitions written in a subset of P4, a database of protocol headers and parser rules.

The KUNAI compiler first interprets the layer-chain: it checks that each protocol may connect to the next according to the definitions and, when the same protocol appears more than once, binds a label to each layer slot. It then resolves field references against the protocol definitions, obtaining the layer, offset, and bit width of each field. When a reference targets an auxiliary header list or an option, the scan target and the information needed to walk it are also recorded in the IR. References to non-existent fields, a `proto.field` reference when that protocol appears more than once, and comparisons whose type or bit width do not match are rejected at this analysis stage.

The output of the KUNAI compiler is eBPF bytecode that evaluates the filter condition against a packet and returns a match/reject result. At run time, a BPF program placed at an attach point such as XDP [11], tc, or fentry/fexit embeds this bytecode. **Under this division**, the KUNAI compiler handles filter-condition analysis and bytecode generation, while **the host BPF program** handles attach-point-specific context access and follow-on actions such as capture or drop.

3.2 What the DSL Expresses

The KUNAI DSL describes a packet's layer structure and field conditions as a single filter expression. **An excerpt of the grammar follows.**

```
filter ::= layer-chain where-clause?
layer-chain ::= layer ('/' layer)*
layer ::= layer-atom quantifier? predicate*
layer-atom ::= proto-name ('@' label)?
              | '(' layer ('/' layer)+ ')'
quantifier ::= '?' | '+' | '*' | '{' INT '}' | '{' INT ' ', INT '}'
predicate ::= '[' field-ref op value ']'
where-clause ::= 'where' bool-expr
bool-expr ::= comparison | quant-atom
```

```
| '(' bool-expr ')' | bool-expr bool-op bool-expr
comparison ::= field-ref op value
quant-atom ::= ('any' | 'all') '(' bool-expr ')'
field-ref ::= ident index? ('.' ident index?)*
```

A layer-chain lists, with /, the order in which protocols appear in the packet, and can also express optional layers, repetition, and choice. The where clause is a condition over the fields of layers in the chain and of auxiliary headers and lists. KUNAI resolves a field reference into one of the forms in Table 1; there, `<stack>` is not a literal but the name of an auxiliary header list given in the protocol definition, such as segments or exts.

Reference form (example)	Meaning
<code>proto.field(tcp.dport)</code>	primary field of a protocol unique in the chain
<code>label.field(inner.dst)</code>	field of a layer carrying a label (@inner)
<code>proto.aux.field</code>	field of an optional or auxiliary header
<code>proto.<stack>[N].field</code>	static index into an auxiliary header list
<code>proto.<stack>.field</code>	element being walked in any()/all()
<code>proto.options.NAME.field</code>	TCP / IPv4 option lookup

Table 1: Field-reference forms KUNAI resolves.

KUNAI interprets a field reference inside a bracket predicate as relative to that layer. In the where clause, references name the layer explicitly, as in `proto.field` or `label.field`.

When the same protocol appears more than once in a chain, `proto.field` alone does not determine which layer to read. For example, when a chain contains both an outer and an inner **IPv4**, `ipv4.dst` does not uniquely identify which destination field is meant. KUNAI rejects such a reference before code generation and requires a label-qualified reference. A filter on the inner IPv4 destination of a GTP-U tunnel, for example, is written as follows.

```
eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst ==
10.0.0.1
```

Here @outer and @inner distinguish the two IPv4 layers. Because `inner.dst` uniquely names the destination field of the inner IPv4, KUNAI can determine which field to read even though the same `ipv4` header appears twice.

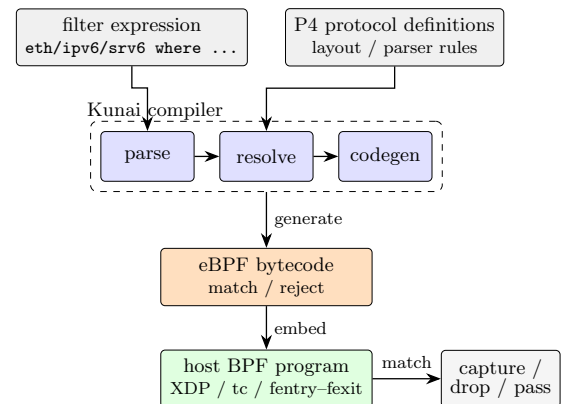


Figure 1: Overview of KUNAI. From a filter expression and P4 protocol definitions, the compiler produces match/reject eBPF bytecode through parse/resolve/codegen, which a host BPF program embeds at an attach point.

Variable-length lists are handled with `any()` and `all()`. A packet whose SRv6 segment list contains a given address, for example, is written as follows.

```
eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)
```

The scan target of `any()` and `all()` is the auxiliary header list referenced without an index inside the expression, and there must be exactly one such target list. The same list may be referenced several times; for example, `any(srv6.segments.addr == fc00::1 or srv6.segments.addr == fc00::2)` is valid. **Two different lists, however, cannot be walked within a single quantifier.** KUNAI evaluates the inner condition against each element of the list. The walk requires a field giving the element count, the width of each element, a way to advance to the next element, and a stop condition; the protocol definition supplies these. TCP options and GTP extension headers are handled the same way when the protocol definition carries this information.

In this way, the KUNAI DSL specifies, within a filter expression, the order in which headers appear, the disambiguation of repeated protocols, and the target of a variable-length walk. Code generation uses this information to translate each field access into an offset from the layer's start and a bit width.

3.3 Protocol Definitions via a P4 Subset

KUNAI uses a subset of the P4 language to describe protocol headers and parser rules [17]. Because P4 already has syntax for protocol headers and parser transitions, existing P4 parser descriptions can serve as a reference when writing a protocol definition. Using P4 also avoids implementing and maintaining the syntax checker, parser, tooling, and specification that a bespoke protocol-description language would require.

The P4 subset KUNAI handles is limited to the parts that describe header layout and parser transitions. **The listing below** is the SRv6 protocol definition; the `any()` example in §3.2 and filter F8 in §5 use it.

```
header srv6_h {
  bit<8> next_header;
  bit<8> hdr_ext_len; // in 8-byte units
  bit<8> routing_type; // 4 = SRH
  bit<8> segments_left;
  bit<8> last_entry;
  bit<8> flags;
  bit<16> tag;
}

header srv6_seg_h { bit<128> addr; }
const bit<8> SRV6_IPV6_NEXT_HEADER = 43;
parser SRv6Parser(packet_in pkt, out srv6_h hdr,
  @kunai_layout[after=primary]
  @kunai_stack_count[field=last_entry, offset=1]
  out srv6_seg_h[8] segments) {
  state start {
    pkt.extract(hdr);
    transition select(hdr.routing_type) {
      4: skip_segments; default: reject;
    }
  }
  state skip_segments {
    pkt.advance(((bit<32>)(hdr.hdr_ext_len & 0x0F)) << 6);
    transition accept;
  }
}
```

This definition supplies the information the KUNAI compiler needs through three constructs. First, the header declarations give the offset and bit width of each field. Second, the `const` and `transition select` give the dispatch from IPv6 to SRv6 and the acceptance test on `routing_type`. Third, the annotations give the walk metadata for the variable-length part: `@kunai_layout` indicates that the segments list begins immediately after **the primary header**, and `@kunai_stack_count` that the element count is read as `last_entry + 1`; the bounded walk of §4 uses these. The capacity 8 of `srv6_seg_h[8]` becomes the compile-time upper bound on the number of iterations. The `pkt.advance` in `skip_segments` caps the advance of the variable-length part at 120 bytes through a mask, exposing a static bound to the verifier.

In this way, a protocol definition describes layout, dispatch, and walk metadata within P4 syntax. Adding a protocol definition extends the set of supported protocols without changing the compiler itself.

4 VERIFIER-CONSTRAINED CODE GENERATION

This section describes how KUNAI lowers a filter expression into the packet accesses, loops, and bounds checks that the Linux eBPF verifier accepts. In a filter that contains multi-level encapsulation or variable-length structure, the start offset of a **following** header changes per packet. Reading the packet after computing this offset at run time leaves the read range unprovable to the verifier, and the bytecode is rejected. The code generation in this section evaluates the filter condition while avoiding this rejection.

KUNAI's code generation lowers the filter condition into a verifier-accepted form in four steps, described in §4.1 and §4.2. First, it records at run time the start offset of each following header that changes per packet. Second, it walks variable-length structures such as SRv6 segments and TCP options under a bound. Third, it translates a field access such as `inner.dst` or `tcp.dport` into a load that uses the resolved layer, offset, and width. Fourth, it places **a verifier-checkable bounds check** before every packet load.

4.1 Recording Header Start Offsets Explicitly

In a chain where the start offset of a **following** header changes per packet, a naive lowering that reads a field at a fixed offset from the start of the packet fails. KUNAI handles this by recording each header's start offset at run time and reading a field as a relative offset from that start.

Consider again the GTP-U inner-IPv4 filter of §3.2. In this filter, `inner.dst` is not at a fixed offset from the start of the packet: the start of the inner **IPv4** changes with the length of the outer IPv4 header, the presence of GTP optional fields, and **the shape** of the encapsulation.

Lowering such a chain naively produces two failures. First, when a naive code generator resolves the inner header to a fixed compile-time offset, it reads the wrong bytes whenever the variable-length parts before it—GTP optional fields, Geneve options, IPv4 options, or SRv6 segments—are non-empty, and returns the wrong match/reject. For example, resolving `eth/ipv4@outer/udp/geneve/eth/ipv4@inner/tcp` where `inner.dst` with a fixed Geneve length reads a position shifted by the options on a packet with `opt_len >`

0. Second, loading at a run-time offset directly against the native-XDP packet pointer (`PTR_TO_PACKET`) leaves the verifier unable to prove statically that the read stays within `data_end`, and the bytecode is rejected. Both failures have the same cause: after a variable-length layer, each layer's start offset depends on run time.

KUNAI detects this boundary at the resolve stage and marks every layer after one with a variable-length layout as a run-time offset. The generated eBPF bytecode then records each header's start offset into a stack slot as it traverses the header at run time. Figure 2 illustrates this offset recording and a bounds-checked load that uses the recorded offset.

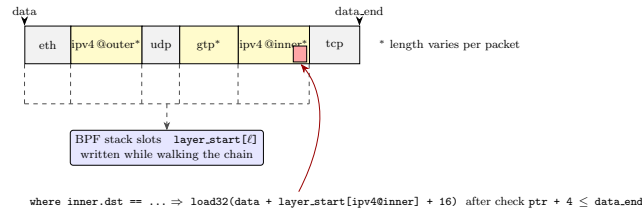


Figure 2: Recording each header's start offset into a stack slot, and reading `inner.dst` as a bounds-checked load at `layer_start + field offset`.

KUNAI generates a field access not as a fixed offset from the start of the packet but as the sum of the start offset of the layer the field belongs to and the field offset. As a result, even when the same protocol appears more than once, as with outer and inner, KUNAI reads the field the same way using the label-resolved layer slot.

Every field access has the same **shape** as this example: KUNAI emits, in order, the load of the layer's start offset, the addition of the field offset and width, the check that the access does not exceed `data_end`, the load, and the typed comparison. Because type-mismatched comparisons and a `proto.field` reference under a repeated protocol are already rejected at the analysis stage of §3.1, code generation only needs to emit the bounds-checked load. KUNAI thus generates, even for a chain whose following-header start changes at run time, bytecode that reads each field relative to the recorded start offset and places a bounds check before each read, so the verifier can confirm that the read stays within the packet.

4.2 Lowering Variable-Length Structures to Bounded Bytecode

In variable-length structures such as SRv6 segments and TCP options, the element count or the length of each element changes per packet, so they cannot be walked at a fixed offset. KUNAI puts the walk into a verifier-accepted form by translating it into bytecode with a compile-time constant bound.

Consider again the SRv6 segment-list filter of §3.2. Because the SRv6 segment list varies in element count per packet, the filter must walk the list in order rather than compare at a fixed offset.

KUNAI generates the walk of a variable-length structure as one of two kinds of bytecode. A parser-machine self-loop and an unbounded chain quantifier are translated into a `bpf_loop` callback with a constant bound [6]. A quantification over an auxiliary header list with `any()/all()` is translated into a fixed unrolling up to the

maximum count. For SRv6 segments, the `@kunai_stack_count` annotation in the listing of §3.3 gives the actual element count as `last_entry + 1`, so each iteration checks that it does not exceed that count.

Algorithm 1 shows this walk; the code generator instantiates it per protocol.

Algorithm 1 Bounded element walk.

Require: `data`, `data_end`, `base`, `stride`, `count`, `cap`, `target`

```

1: matched ← false
2: for i ← 0 to cap − 1 do                                ▷ cap: compile-time bound
3:   if i ≥ count then break                                ▷ skip slots past the count
4:   ptr ← data + base + i · stride
5:   if ptr + stride > data_end then break                  ▷ bounds check
6:   if load(ptr) = target then matched ← true; break
7: return matched

```

For the SRv6 filter above, `base` = `srh_start` + `SRH_FIXED_SIZE`, `stride` = 16, `count` = `last_entry` + 1, and `target` = `fc00::1`, while `cap` is the compile-time constant bound.

KUNAI generates the TCP-options walk with the same structure as Algorithm 1. Because TCP options differ in length per option and a reference such as `tcp.options.MSS.value` cannot be located at a fixed offset, `stride` is not a fixed value but the length of each option, and the **cursor** advances while reading in order. The walk stops when the cursor reaches the end of the options, when an END option is read, or when the iteration bound is reached. It also stops when it detects a malformed option, such as `len < 2` or `cursor + len > options_end`. An option-specific field is loaded only after checking that its option header does not cross the end of the packet window.

The KUNAI compiler treats the bound of a variable-length walk as a constant determined by the protocol definition and the code-generation settings. SRv6 segments and TCP options differ in structure, but KUNAI generates both as bounded walks and, in either the fixed-unrolling or the `bpf_loop` form, places a bounds check before each packet load. In this form, the verifier can confirm both the loop bound and the range of every packet access.

So that the generated bytecode is accepted across kernel versions, KUNAI avoids instructions available only on newer versions. For byte-order conversion, for example, it uses the `BPF_END` instruction, available on older versions, rather than the `BSWAP` instruction introduced in 6.6 [13].

5 EVALUATION

We evaluate KUNAI from three angles. First, whether the packet structures KUNAI targets can be written as filter expressions (§5.2). Second, whether the generated bytecode is accepted by the eBPF verifier and matches/rejects as expected (§5.3). Third, the instruction count and run-time cost of the generated bytecode (§5.4).

5.1 Method and Filter Set

The evaluation uses ten filters, **F1–F10**. **F1–F6** target basic protocol headers such as Ethernet, VLAN, IPv4/IPv6, TCP, and ICMP, while **F7–F10** target nested headers and variable-length structures whose start cannot be fixed at a constant offset. The expressiveness gap this

paper addresses appears in F7–F10, ~~so we place them at the center of the evaluation.~~ Table 2 lists each filter’s KUNAI expression and the raw eBPF instruction count of the bytecode KUNAI generates, together with the count for the same filter compiled from pcap-filter to eBPF through cBPF using cbpf [4]. We refer to this latter count as the pcap-filter baseline in the rest of the evaluation.

We pinned the KUNAI instruction count of each filter as an expected value in a regression test; if a compiler change alters a count unintentionally, the difference from the table value surfaces as a test failure.

To exercise the DSL syntax more broadly, we prepared a **separate regression suite of 68 filter expressions**. The suite covers bracket predicates, arithmetic in the where clause, quantified layers, heterogeneous-size alternations, label disambiguation, and aux-walks, spanning a wider syntactic range than F1–F10. Its verifier acceptance and per-packet match/reject correctness are reported in §5.3.

To evaluate these filters, we embedded the generated bytecode in a host BPF program and placed it at the XDP and tc attach points. ~~As described in §3.1, the KUNAI compiler produces attach-point-independent bytecode, and the host BPF program supplies the attach-point-specific packet context and entry/exit handling.~~ The remaining subsections use this filter set and regression suite throughout.

5.2 Expressiveness

We compared whether F1–F10 can be written in pcap-filter, the filter language that runs in the kernel, and in KUNAI. Because pcap-filter has byte-offset access, a hand-written condition can sometimes be built for a specific packet layout. It does not, however, provide as filter syntax the order of headers, an arbitrary element of a variable-length list, repeated occurrences of the same protocol, or a named reference to an inner header. F1–F6 were writable in both; only F5 (QinQ) behaved in a capture-environment-dependent way in pcap-filter. The two diverge at F7–F10. The inner IPv4 of F7 (GTP-U) and F9 (Geneve) sits at no fixed offset, shifted per packet by GTP optional fields and extension headers or by a Geneve inner Ethernet header; F8 (SRv6) requires looping over an arbitrary segment-list element; and F10 (TCP MSS) requires an option walk rather than a byte-offset approximation. None of these is expressible as pcap-filter syntax, whereas KUNAI writes all of F1–F10 directly through the labeled layer-chains, any()/all() quantifiers, and option-field walks listed in Table 2.

5.3 Verifier Acceptance and Correctness

We made two checks of the code generation of §4: first, that the generated bytecode is accepted by the eBPF verifier; second, that the accepted bytecode matches and rejects as expected.

For acceptance, we loaded the bytecode generated from F1–F10 and the regression suite onto six kernels (Linux 6.1, 6.6, 6.12, 6.15, 6.18, 7.0) at both the XDP and tc attach points. The instructions and helpers the verifier accepts differ across kernel versions, but because KUNAI avoids version-dependent instructions as described in §4, the same bytecode was accepted unchanged on all six versions.

At tc, because the kernel extracts the outer VLAN tag into skb metadata, KUNAI rejects at compile time a filter that reads

VLAN/QinQ as a byte parser. This rejection is a compile-time restriction, not a verifier rejection. We therefore excluded 2 of F1–F10 and 6 of the regression suite from the tc loads. After this exclusion, we performed 888 loads in total across six kernels $\times$ 2 hosts, all of which were accepted.

Next, we ran a packet-level test that feeds the generated bytecode actual packets and checks that it matches the expected packets and rejects the rest. For each protocol we constructed matching, non-matching, and malformed packets; for Geneve, for example, we varied opt_len between 0 and a positive value to confirm that the inner offset past the skipped options resolves correctly, and we included truncated and malformed frames throughout. In every case the bytecode returned the expected match/reject. We ran the same test on the regression suite, which also matched and rejected as expected.

These two checks show that the code generation of §4 lowers the target filters into bytecode accepted across six kernels $\times$ 2 hosts and traverses packet headers correctly.

5.4 Instruction Count and Run-Time Cost

We first evaluate a filter’s cost by the instruction count of the generated eBPF bytecode, listed per filter in Table 2. The instruction count is the static size of the bytecode and is comparable independent of the measurement environment; a 64-bit immediate load is counted as two instructions for both KUNAI and pcap-filter. On the F1–F4 and F6 filters that pcap-filter can also express, KUNAI generated 2.7 $\times$ to 14.7 $\times$ as many instructions, the largest gap arising at the IPv6 prefix match of F3.

We measured whether this instruction-count difference shows up as run-time cost on a datapath that sends actual traffic. We connected a traffic generator directly to the device under test (DUT) over 100 GbE, and attached on the DUT a native XDP program that evaluates the filter and returns XDP_DROP on every packet. The DUT is an Intel Xeon Platinum 8362 with an Intel E810 NIC, Linux 6.15, and the BPF JIT enabled. As in an operational fentry/tc observer, this program copies the head of the packet into a per-CPU scratch window before evaluating the filter. This copy alone lowered the processing rate by about 6% even for an accept-all program with no filter. We therefore measured each filter’s cost as the processing-rate reduction relative to the accept-all program that performs the same copy, isolating the filter’s own cost from the fixed copy cost.

Figure 3 shows the per-filter processing-rate reduction. We measured at the 64-byte line rate and repeated each cell ten times; every standard deviation was at most 0.4. The reduction for the basic filters F1–F6 was 0–1.8%, near 0% for F6, which tests the ICMP type. The KUNAI-only filters F7–F10 were 4.0–6.1%, with F7—which entails a full chain walk—the largest at 6.1%. Even this F7 stayed within the 6.3% fixed cost of the window copy alone. For F1, writable in both KUNAI and pcap-filter, the difference between the two was only 0.7%.

This datapath result agreed with a per-packet run-time measurement via BPF_PROG_TEST_RUN through the syscall. Taking the filterless accept-all as the baseline, the increment of each filter over its 608–610 ns was at most +13 ns/pkt, a difference of less than 3% from the baseline. The difference between KUNAI and pcap-filter

ID	KUNAI expression	raw insns	
		KUNAI	pcap-filter
F1	eth/ipv4/tcp where tcp.dport == 443	199	42
F2	eth/ipv4[src==10.0.0.0/8]/tcp where tcp.dport == 80	209	36
F3	eth/ipv6[src==2001:db8::/32]/tcp	338	23
F4	eth/vlan[tcid==100]/ipv4/tcp where tcp.dport == 80	215	51
F5	eth/qinq/vlan/ipv4/tcp where tcp.dport == 80	217	—
F6	eth/ipv4/icmp where icmp.type == 8	85	31
F7	eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst == 10.0.0.1	405	—
F8	eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)	450	—
F9	eth/ipv4@outer/udp/geneve/eth/ipv4@inner/tcp where inner.dst == 10.0.0.1	323	—
F10	eth/ipv4/tcp where tcp.options.MSS.value == 1460	231	—

Table 2: Evaluation filter set and raw eBPF instruction counts. F7–F10 target nested headers and variable-length structures. The pcap-filter column is the eBPF compiled from each filter via cBPF; — marks those pcap-filter cannot express.

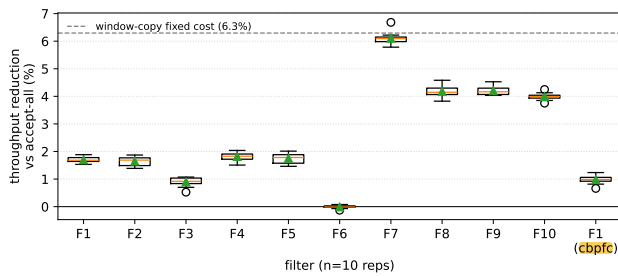


Figure 3: Datapath processing-rate reduction for F1–F10 (relative to accept-all, box plot over $n=10$). The heaviest filter F7 ($\approx 6\%$) stays within the window-copy fixed cost (6.3%, dashed). Green triangles are means.

was at most comparable to the measurement standard deviation, confirming that their run times are equivalent.

The reason the up-to-14.7 $\times$ instruction-count difference does not appear in per-packet run time is that the total per-packet cost is dominated by fixed costs such as the window copy, the drop, and NAPI processing, with filter evaluation only a small part of it. In sum, the instruction-count difference does not appear in the per-packet run time of this measurement setup.

6 LIMITATIONS

KUNAI is a DSL specialized for packet filtering, not a general-purpose eBPF language. Stateful filtering, deep packet inspection, and aggregation are out of scope. P4 is used only to define header layout and parser transitions; pipeline execution mechanisms such as actions and tables are not handled.

The filter expression itself also has current limitations. The current code generator does not handle, for example, a bitmask comparison that is not byte-aligned, such as a SYN test on TCP flags, or a walk of a variable label following a quantifier. The expressiveness of KUNAI and pcap-filter is therefore not a **containment relation**: each has conditions the other cannot write.

What we showed for verifier acceptance is an empirical confirmation, not a formal guarantee. We confirmed acceptance and matching on six kernels for F1–F10 and the regression suite, but

acceptance for an arbitrary KUNAI expression on an arbitrary kernel version is future work. Also, because a variable-length walk is translated into bounded bytecode, the number of elements that can be walked has a compile-time bound, and a filter that uses `bpf_loop` is subject to a kernel-version lower bound.

The correctness of the generated bytecode depends on the correctness of the protocol definition. The parse check by p4c verifies P4 syntax and does not guarantee that field offsets or parser transitions are correct with respect to the relevant RFC. The current definitions also have coverage limits: Geneve option TLVs, for example, are not exposed as filter targets, and the tc host rejects a filter that reads VLAN/Qinq as packet bytes (§5.3). These extensions are future work.

7 CONCLUSION

We presented KUNAI, a DSL and compiler that makes packet-filter conditions involving multi-level encapsulation, repeated occurrences of the same protocol, and walks over variable-length lists and options executable inside the Linux kernel. KUNAI describes these conditions as a layer-chain and condition expressions, and performs name resolution, type checking, and offset computation through protocol definitions based on a P4 subset. Its code generation—recording each layer’s start offset, bounding walks by a constant, and emitting type-based bounds-checked loads—transforms a filter expression into the packet accesses and loops the verifier accepts.

Through a series of evaluations, we confirmed that this approach holds as intended. Filter conditions over multi-level encapsulation and variable-length structures, which pcap-filter cannot write as syntax, are expressible in KUNAI; the same generated bytecode is accepted unchanged by the verifier across six kernel versions from Linux 6.1 to 7.0; and it matches and rejects as expected. This expressiveness entails a static instruction-count increase of up to 14.7 $\times$, but we showed by measurement that this difference does not appear in the per-packet run-time cost on a line-rate datapath, and that for filters writable in both languages the cost is equivalent. KUNAI thus makes the filter conditions pcap-filter could not express executable on the Linux kernel datapath, achieving flexible packet filtering beyond what prior approaches allowed.

REFERENCES

- [1] 3GPP. 2023. TS 29.281: GPRS Tunnelling Protocol User Plane (GTPv1-U). 3GPP

- Technical Specification.
- [2] Ivano Cerrato and Fulvio Risso. 2018. Enabling Precise Traffic Filtering based on Protocol Encapsulation Rules. *Computer Networks* (2018).
- [3] Luigi Ciminiera, Mario Baldi, Riccardo Sisto, and Fulvio Risso. 2010. Tunnel-aware Filter Languages: NetPFL. In *Proc. IEEE GLOBECOM*.
- [4] Cloudflare. 2021. cbpf: a cBPF-to-eBPF compiler. <https://github.com/cloudflare/cbpf>.
- [5] eBPF Docs. 2024. Loops in BPF. <https://docs.ebpf.io/linux/concepts/loops/>.
- [6] eBPF Docs. 2024. bpf_loop helper function. https://docs.ebpf.io/linux/helper-function/bpf_loop/.
- [7] W. Eddy. 2022. Transmission Control Protocol (TCP). RFC 9293.
- [8] C. Filsfils et al. 2020. IPv6 Segment Routing Header (SRH). RFC 8754.
- [9] Elazar Gershuni et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proc. ACM PLDI*.
- [10] J. Gross, I. Ganga, and T. Sridhar. 2020. Geneve: Generic Network Virtualization Encapsulation. RFC 8926.
- [11] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proc. ACM CoNEXT*.
- [12] Marco Leogrande, Fulvio Risso, and Luigi Ciminiera. 2015. Modeling Complex Packet Filters with Finite State Automata. *IEEE/ACM Transactions on Networking* (2015).
- [13] Linux Kernel Documentation. 2024. BPF Instruction Set Architecture (ISA). <https://docs.kernel.org/bpf/standardization/instruction-set.html>.
- [14] Linux Kernel Documentation. 2024. eBPF Verifier. <https://www.kernel.org/doc/html/latest/bpf/verifier.html>.
- [15] Steven McCanne and Van Jacobson. 1993. The BSD Packet Filter: A New Architecture for User-level Packet Capture. In *Proc. USENIX Winter Technical Conference*.
- [16] Jeffrey C. Mogul, Richard F. Rashid, and Michael J. Accetta. 1987. The Packet Filter: An Efficient Mechanism for User-level Network Code. In *Proc. ACM SOSP*.
- [17] P4 Language Consortium. 2023. P4-16 Language Specification. <https://p4.org/p4-spec/docs/P4-16-v1.2.5.pdf>.
- [18] P4 Language Consortium. 2024. P4C Documentation: eBPF Backend. https://p4lang.github.io/p4c/ebpf_backend.html.
- [19] Fulvio Risso and Mario Baldi. 2006. NetPDL: An Extensible XML-based Language for Packet Header Description. *Computer Networks* (2006).
- [20] Noa Sultana et al. 2024. A Survey on Packet Filtering. *ACM SIGCOMM Computer Communication Review* (2024).
- [21] tcpdump.org. 2024. pcap-filter(7) manual page. <https://man7.org/linux/man-pages/man7/pcap-filter.7.html>.
- [22] William Tu et al. 2018. p4c-xdp: Compiling P4 to XDP. In *Linux Plumbers Conference*.
- [23] Wireshark Foundation. 2024. Wireshark Display Filter Reference. <https://www.wireshark.org/docs/dfref/>.